

Documentación Técnica del Proyecto

Estación Meteorológica Educativa Integrada (Arduino + Raspberry Pi + MySQL)

Proyecto: Monitoreo Ambiental
STEM

Nivel: Preparatoria /
Bachillerato

Arquitectura: Arduino (Adquisición) & Python/MySQL
(Procesamiento)

1. Resumen General del Proyecto

Descripción: Este proyecto consiste en una estación meteorológica educativa diseñada y programada por estudiantes de preparatoria con el objetivo de fomentar el aprendizaje científico, el monitoreo ambiental y el desarrollo tecnológico.

El sistema recopila datos atmosféricos y ambientales en tiempo real utilizando un microcontrolador Arduino para la captura de variables analógicas/digitales y una computadora de placa única (Raspberry Pi/Linux) para el cómputo avanzado, almacenamiento persistente en base de datos MySQL y registro local CSV.

2. Objetivos y Alcance

- **Educativo & STEM:** Demostrar cómo los sistemas embebidos de bajo costo pueden utilizarse para monitoreo climático, experimentación científica e investigación en el aula.
- **Redundancia y Robustez:** Implementar tolerancia a fallos en sensores críticos (como conmutación automática entre sensores DHT11 primario y secundario).
- **Medición Multivariable:** Medición en exteriores y sombra de temperatura, humedad, presión atmosférica, velocidad del viento, precipitación pluvial acumulada y calidad del aire (PPM).
- **Integración de Software:** Protocolo de comunicación Serial entre microcontrolador y servidor Python con persistencia en MySQL (ba1un_canan).

3. Arquitectura del Hardware y Sensores

Componente / Sensor	Interfaz / Pin	Variable Medida	Función / Observaciones
Arduino Microcontroller	Serial USB (9600 baud)	Adquisición en tiempo real	Lee sensores de campo, procesa pulsos e interrupciones, envía trama CSV por Serial.
BME280 (I2C)	I2C (0x76 / Bus 1)	Temp (°C), Presión (hPa), Humedad (%)	Sensor de alta precisión presente en módulo Arduino y servidor Raspberry Pi.
DHT11 (x2)	Digital Pin 5 (Pri) / Pin 8 (Sec)	Temperatura / Humedad Relativa	Sistema de respaldo con reinicio de alimentación mediante transistores/relés (Pins 7 y 4).
Anemómetro Analógico	Pin Analógico A3	Velocidad del Viento (m/s o km/h)	Ecuación de calibración lineal aplicada: $vel = 0.78 \times I1 - 5.46$.
Pluviómetro de Balancín	Pin Digital 3 (INT)	Precipitación acumulada (mm)	Mapeo por interrupción (FALLING). Calibrado a 0.3 mm/pulso con anti-rebote (300 ms).
Sensor de Lluvia	Pin Analógico A2	Detección cualitativa de lluvia	Identifica inicio de lluvia (≥ 200) o lluvia definitiva (≥ 600).
MQ-135	Pin Analógico A1	Calidad del aire / Gases (PPM)	Cálculo de concentración de gases compensado por temperatura y humedad.

4. Código Arduino (`estacion\_meteorologica.ino`)

Arduino C++ - Captura y Control de Sensores

```
#include <MQ135.h>
#include <Adafruit_BME280.h>
#include <Wire.h>
#include <DHT.h>

// Definición de Pines
const int pinV = 7;
const int pinV2 = 4;
const int Ane = A3;
const int sensorA = A2;
const int pluv = 3;
#define PIN_MQ135 A1

#define DHTPIN 5
#define DHTPIN1 8
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);
DHT dht1(DHTPIN1, DHTTYPE);
Adafruit_BME280 bme;
MQ135 mq135_sensor(PIN_MQ135);

// Variables Pluviómetro
volatile int contadorPluv = 0;
volatile unsigned long ultimoPulso = 0;
volatile unsigned long ultimoPulsoReal = 0;
const float mmPulso = 0.3;
const int umbralTiempo = 300; // Anti-rebote (ms)

bool advertencial = false;
```

```

bool advertencia2 = false;

void setup() {
  Serial.begin(9600);
  dht.begin();
  dht1.begin();
  Wire.begin();

  bme.begin(0x76);

  pinMode(pinV, OUTPUT);
  pinMode(pinV2, OUTPUT);
  pinMode(Ane, INPUT);
  pinMode(sensorA, INPUT);
  pinMode(pluv, INPUT_PULLUP);

  digitalWrite(pinV, HIGH);
  digitalWrite(pinV2, HIGH);

  attachInterrupt(digitalPinToInterrupt(pluv), contarLluvia, FALLING);
}

void contarLluvia() {
  unsigned long ahora = millis();
  if (ahora - ultimoPulso > umbralTiempo) {
    contadorPluv++;
    ultimoPulso = ahora;
    ultimoPulsoReal = ahora;
  }
}

void loop() {
  float lecturaA = analogRead(sensorA);

  noInterrupts();
  int pulsos = contadorPluv;
  unsigned long ultimo = ultimoPulsoReal;
  interrupts();

  if (millis() - ultimo > 30000UL) {
    noInterrupts();
    contadorPluv = 0;
    interrupts();
    pulsos = 0;
  }

  float mmTotal = pulsos * mmPulso;

  // Manejo de Respaldo DHT11
  float tem1 = dht.readTemperature();
  float hum1 = dht.readHumidity();

  if ((isnan(tem1) || isnan(hum1)) && !advertencia1) {
    advertencia1 = true;
    Serial.println("Reiniciando DHT11 primario...");
    digitalWrite(pinV, LOW);
    delay(2000);
    digitalWrite(pinV, HIGH);
    delay(2000);
    tem1 = dht.readTemperature();
    hum1 = dht.readHumidity();
  } else if ((isnan(tem1) || isnan(hum1)) && advertencia1 && !advertencia2) {
    Serial.println("Cambiando a DHT11 de repuesto...");
    advertencia2 = true;
    digitalWrite(pinV2, HIGH);
    tem1 = dht1.readTemperature();
    hum1 = dht1.readHumidity();
  }
}

```

```

} else if (isnan(tem1) || isnan(hum1)) {
    Serial.println("PROBLEMA CON DHT11: REVISAR");
} else {
    digitalWrite(pinV, HIGH);
    digitalWrite(pinV2, HIGH);
}

float temp = bme.readTemperature();
float pres = bme.readPressure() / 100.0;

int l1 = analogRead(Ane);
float vel = 0.78 * l1 - 5.46;
if (vel < 0) vel = 0; // Evitar valores negativos por descalibración en reposo

float ppm = mq135_sensor.getCorrectedPPM(temp, hum1);

// Formato Serial CSV: Temp_BME, Temp_DHT, Presion, Vel_Viento, Precipitacion, Humedad_DHT, PPM
Serial.print(temp); Serial.print(",");
Serial.print(tem1); Serial.print(",");
Serial.print(pres); Serial.print(",");
Serial.print(vel); Serial.print(",");
Serial.print(mmTotal); Serial.print(",");
Serial.print(hum1); Serial.print(",");
Serial.println(ppm);

delay(5000);
}

```

5. Código Python (`programa\_6.py`)

Python 3 - Cliente de Procesamiento, CSV y MySQL

```

import time
from datetime import datetime
import math
import os
import serial
from smbus2 import SMBus
from bme280 import BME280
import mysql.connector

# =====
# CÁLCULOS METEOROLÓGICOS
# =====

def punto_rocio(temp, hum):
    a = 17.27
    b = 237.7
    alpha = ((a * temp) / (b + temp)) + math.log(hum / 100.0)
    return (b * alpha) / (a - alpha)

def sensacion_termica(temp, hum):
    return (-8.784695 +
            1.61139411 * temp +
            2.338549 * hum -
            0.14611605 * temp * hum -
            0.012308094 * (temp**2) -
            0.016424828 * (hum**2) +
            0.002211732 * (temp**2) * hum +
            0.00072546 * temp * (hum**2) -
            0.00003582 * (temp**2) * (hum**2))

# Configuración de Sensores
bus = SMBus(1)

```

```

bme280 = BME280(i2c_dev=bus)

# Conexión Base de Datos
db_pass = os.environ.get("DB_PASSWORD", "3and1D0*")
db = mysql.connector.connect(
    host="localhost",
    user="root",
    password=db_pass,
    database="balun_canan"
)
cursor = db.cursor()

# Conexión Serial Arduino
try:
    ser = serial.Serial('/dev/ttyACM0', 9600, timeout=2)
    time.sleep(2)
except Exception as e:
    print(f"❌ Error al abrir puerto Serial: {e}")
    ser = None

def leer_arduino():
    if ser is None:
        return None, None, None, None, None, None, None

    try:
        line = ser.readline().decode('utf-8').strip()
        if line:
            valores = line.split(',')
            if len(valores) == 7:
                temp_A = float(valores[0])
                temp_ext_A = float(valores[1])
                presion_A = float(valores[2])
                anemometro = float(valores[3])
                precipitacion = float(valores[4])
                humedad_A = float(valores[5])
                ppm_A = float(valores[6])
                return temp_A, temp_ext_A, presion_A, anemometro, precipitacion, humedad_A, ppm_A
            else:
                print(f"⚠️ Trama incompleta. Recibidos: {len(valores)} campos")
    except Exception as e:
        print(f"❌ Error lectura Serial: {e}")

    return None, None, None, None, None, None, None

# Bucle Principal
print('Comienzo de lectura de estación meteorológica...')
contador = 0

with open('datos.csv', 'a', buffering=1) as f:
    while True:
        try:
            temperatura = bme280.get_temperature()
            presion = bme280.get_pressure()
            humedad = bme280.get_humidity()

            dew = punto_rocio(temperatura, humedad)
            heat = sensacion_termica(temperatura, humedad)

            temp_A, temp_ext_A, presion_A, anemometro, precipitacion, humedad_A, ppm_A = leer_arduino()

            now = datetime.now()
            fecha_hora = now.strftime("%Y-%m-%d %H:%M:%S")

            print(f"[{fecha_hora}] Local BME280: {temperatura:.2f}°C | {presion:.2f} hPa | {humedad:.2f}%")
            print(f"    → Rocío: {dew:.2f}°C | Sensación: {heat:.2f}°C")
            print(f"    → Arduino: T={temp_A}, TExt={temp_ext_A}, P={presion_A}, Wind={anemometro},")
            Rain={precipitacion}")

```

```

        if -40 < temperatura < 85 and 300 < presion < 1100:
            f.write(f"{fecha_hora},{temperatura:.2f},{presion:.2f},{humedad:.2f},{dew:.2f},{heat:.2f},"
                    f"{temp_A},{temp_ext_A},{presion_A},{anemometro},{precipitacion},{humedad_A},{ppm_A}"
                    "\n")

            db.ping(reconnect=True)
            sql = ("INSERT INTO Datos_Arduino "
                    "(fecha, temperatura, presion, humedad, punto_rocio, sensacion_termica, "
                    "temp_A, temp_ext_A, presion_A, anemometro, precipitacion, humedad_A, ppm_A) "
                    "VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)")

            valores_db = (fecha_hora, temperatura, presion, humedad, dew, heat,
                           temp_A, temp_ext_A, presion_A, anemometro, precipitacion, humedad_A, ppm_A)

            cursor.execute(sql, valores_db)
            contador += 1
            if contador >= 10:
                db.commit()
                print("✅ Lote de 10 registros guardado en MySQL")
                contador = 0

    except Exception as e:
        print("❌ Error general en bucle:", e)

    time.sleep(60) # Intervalo de lectura sugerido: 60 segundos

```

6. Guía de Instalación y Puesta en Marcha

1. **Configuración en Arduino:** Instalar las librerías MQ135, Adafruit_BME280 y DHT en el IDE de Arduino y compilar el archivo .ino.
2. **Creación de Base de Datos MySQL:**

```

CREATE DATABASE balun_canan;
USE balun_canan;

CREATE TABLE Datos_Arduino (
    id INT AUTO_INCREMENT PRIMARY KEY,
    fecha DATETIME,
    temperatura FLOAT,
    presion FLOAT,
    humedad FLOAT,
    punto_rocio FLOAT,
    sensacion_termica FLOAT,
    temp_A FLOAT,
    temp_ext_A FLOAT,
    presion_A FLOAT,
    anemometro FLOAT,
    precipitacion FLOAT,
    humedad_A FLOAT,
    ppm_A FLOAT
);

```

3. **Ejecución del Servicio en Python:**

```

pip install smbus2 matrix-bme280 mysql-connector-python pyserial
python3 programa_6.py

```